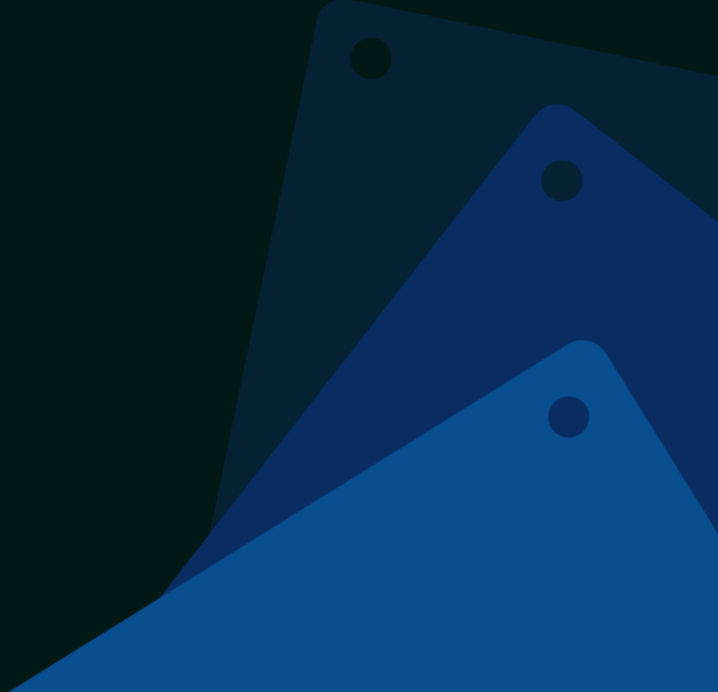


Operating Systems

CHAPTER 6

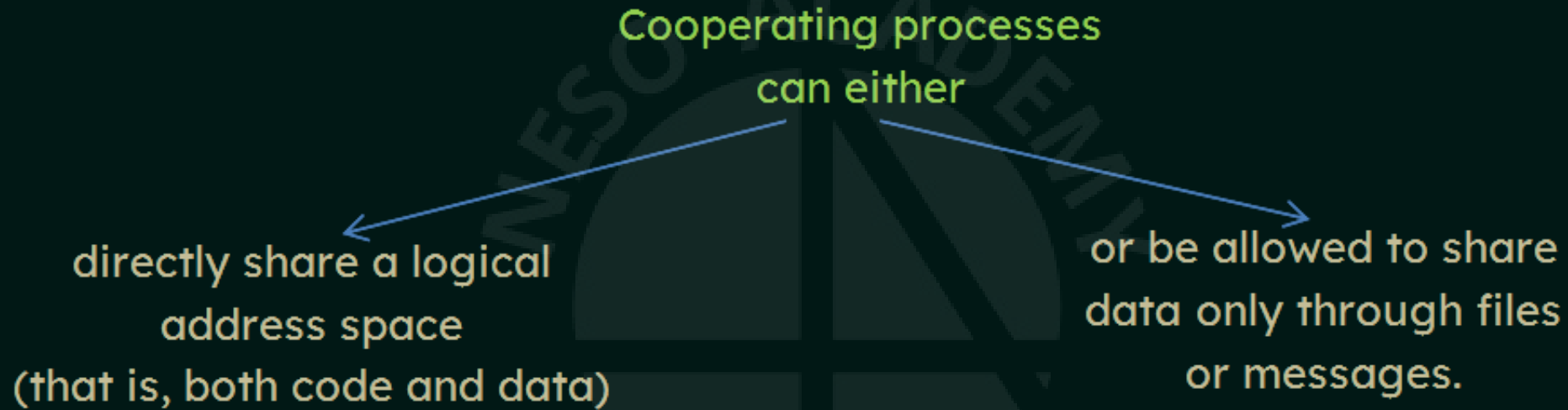
Process Synchronization

Neso Academy



Process Synchronization

A cooperating process is one that can affect or be affected by other processes executing in the system.



Concurrent access to shared data may result in data inconsistency!

In this chapter,
we discuss various mechanisms to ensure –
The orderly execution of cooperating processes that share a logical address space,

So that data consistency is maintained.

counter variable = 0

counter is incremented every time we add a new item to the buffer counter++

counter is decremented every time we remove one item from the buffer counter--

Example

- Suppose that the value of the variable counter is currently 5.
 - The producer and consumer processes execute the statements "counter++" and "counter--" concurrently.
 - Following the execution of these two statements, the value of the variable counter may be 4, 5, or 6!
 - The only correct result, though, is counter == 5, which is generated correctly if the producer and consumer execute separately.
-

"counter++" may be implemented in machine language (on a typical machine) as:

register₁ = counter
register₁ = register₁ + 1
counter = register₁

"counter--" may be implemented in machine language (on a typical machine) as:

register₂ = counter
register₂ = register₂ - 1
counter = register₂

T ₀ :	producer	execute	register ₁ = counter	{ register ₁ = 5 }
T ₁ :	producer	execute	register ₁ = register ₁ + 1	{ register ₁ = 6 }
T ₂ :	consumer	execute	register ₂ = counter	{ register ₂ = 5 }
T ₃ :	consumer	execute	register ₂ = register ₂ - 1	{ register ₂ = 4 }
T ₄ :	producer	execute	counter = register ₁	{ counter = 6 }
T ₅ :	consumer	execute	counter = register ₂	{ counter = 4 }

We would arrive at this incorrect state because we allowed both processes to manipulate the variable counter concurrently.

A situation like this, where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.

Clearly, we want the resulting changes not to interfere with one another. Hence we need **process synchronization**.

The Critical-Section Problem

Consider a system consisting of n processes $\{P_0, P_1, \dots, P_n\}$.

Each process has a segment of code, called a

critical section

in which the process may be changing common variables, updating a table, writing a file, and so on.

When one process is executing in its critical section, no other process is to be allowed to execute in its critical section.

That is, no two processes are executing in their critical sections at the same time.

The critical-section problem is to design a protocol that the processes can use to cooperate.

- Each process must request permission to enter its **critical section**.
- The section of code implementing this request is the **entry section**.
- The critical section may be followed by an **exit section**.
- The remaining code is the **remainder section**.

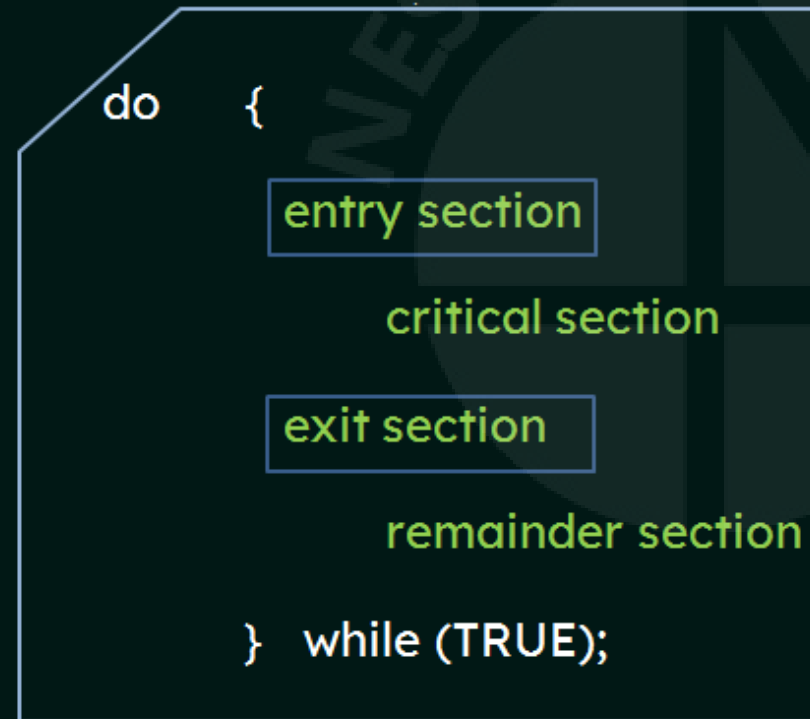


Figure: General structure of a typical process.

A solution to the critical-section problem must satisfy the following three requirements:

1. Mutual exclusion:

If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.

2. Progress:

If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in the decision on which will enter its critical section next, and this selection cannot be postponed indefinitely.

3. Bounded waiting:

There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Peterson's Solution

- A classic software-based solution to the critical-section problem.
- May not work correctly on modern computer architectures.
- However, it provides a good algorithmic description of solving the critical-section problem and illustrates some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting requirements.

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. Let's call the processes P_i and P_j

Peterson's solution requires two data items to be shared between the two processes:

`int turn`

→ Indicates whose turn it is to enter its critical section.

`boolean flag [2]`

→ Used to indicate if a process is ready to enter its critical section.

Structure of process P_i in Peterson's solution

```
do {  
    flag [i] = true ;  
    turn = j ;  
    while ( flag [ j ] && turn == [ j ] ) ;
```

critical section

```
flag [i] = false ;
```

remainder section

```
} while (TRUE) ;
```

Structure of process P_j in Peterson's solution

```
do {  
    flag [ j ] = true ;  
    turn = i ;  
    while ( flag [ i ] && turn == [ i ] ) ;
```

critical section

```
flag [ j ] = false ;
```

remainder section

```
} while (TRUE) ;
```

Test and Set Lock

- A hardware solution to the synchronization problem.
- There is a shared lock variable which can take either of the two values, 0 or 1.
- Before entering into the critical section, a process inquires about the lock.
- If it is locked, it keeps on waiting till it becomes free.
- If it is not locked, it takes the lock and executes the critical section.

```
boolean TestAndSet (boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

Atomic
Operation

The definition of the TestAndSet () instruction

```
do {  
    while (TestAndSetLock (&lock) );  
    // do nothing  
    // critical section  
    lock = FALSE;  
    // remainder section  
} while (TRUE);
```



```
do {  
    while (TestAndSetLock (&lock) );  
    // do nothing  
    // critical section  
    lock = FALSE;  
    // remainder section  
} while (TRUE);
```

Satisfies mutual-exclusion.

Does not satisfy bounded-waiting.

Semaphores

- Semaphore proposed by Edsger Dijkstra, is a technique to manage concurrent processes by using a simple integer value, which is known as a semaphore.
- Semaphore is simply a variable which is non-negative and shared between threads. This variable is used to solve the critical section problem and to achieve process synchronization in the multiprocessing environment.
- A semaphore **S** is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: **wait ()** and **signal ()**.

wait () → **P** [from the Dutch word **proberen**, which means "to test"]

signal () → **V** [from the Dutch word **verhogen**, which means "to increment"]

Definition of wait ():

```
P (Semaphore S) {  
    while (S <= 0)  
        ; // no operation  
    S-- ;  
}
```

Definition of signal ():

```
V (Semaphore S) {  
    S++ ;  
}
```

All the modifications to the integer value of the semaphore in the wait () and signal() operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Types of Semaphores:

1. Binary Semaphore:

The value of a binary semaphore can range only between 0 and 1. On some systems, binary semaphores are known as mutex locks, as they are locks that provide mutual exclusion.

2. Counting Semaphore:

Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

Disadvantages of Semaphores

- The main disadvantage of the semaphore definition that was discussed is that it requires busy waiting.
- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code.
- Busy waiting wastes CPU cycles that some other process might be able to use productively.
- This type of semaphore is also called a spinlock because the process "spins" while waiting for the lock.

To overcome the need for busy waiting, we can modify the definition of the wait () and signal () semaphore operations.

- When a process executes the wait () operation and finds that the semaphore value is not positive, it must wait.
- However, rather than engaging in busy waiting, the process can block itself.
- The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- Then control is transferred to the CPU scheduler, which selects another process to execute.

Deadlocks and Starvation

- The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes.
- The event in question is the execution of a `signal()` operation. When such a state is reached, these processes are said to be **deadlocked**.

P0
<code>wait (S);</code>
<code>wait (Q);</code>
<code>.</code>
<code>.</code>
<code>.</code>
<code>signal (S);</code>
<code>signal (Q);</code>

P1
<code>wait (Q);</code>
<code>wait (S);</code>
<code>.</code>
<code>.</code>
<code>.</code>
<code>signal (Q);</code>
<code>signal (S);</code>

Classic Problems of Synchronization

(The Bounded-Buffer Problem)

The Bounded Buffer Problem (**Producer Consumer Problem**), is one of the classic problems of synchronization.

There is a buffer of n slots and each slot is capable of storing one unit of data.

There are two processes running, namely, **Producer** and **Consumer**, which are operating on the buffer.



- The producer tries to insert data into an empty slot of the buffer.
- The consumer tries to remove data from a filled slot in the buffer.
- The Producer must not insert data when the buffer is full.
- The Consumer must not remove data when the buffer is empty.
- The Producer and Consumer should not insert and remove data simultaneously.

Solution to the Bounded Buffer Problem using Semaphores:

We will make use of three semaphores:

1. m (mutex), a binary semaphore which is used to acquire and release the lock.
2. empty, a counting semaphore whose initial value is the number of slots in the buffer, since, initially all slots are empty.
3. full, a counting semaphore whose initial value is 0.

Producer

```
do {  
    wait (empty); // wait until empty>0  
                    and then decrement 'empty'  
    wait (mutex); // acquire lock  
    /* add data to buffer */  
    signal (mutex); // release lock  
    signal (full); // increment 'full'  
} while(TRUE)
```

Consumer

```
do {  
    wait (full); // wait until full>0 and  
                  then decrement 'full'  
    wait (mutex); // acquire lock  
    /* remove data from buffer */  
    signal (mutex); // release lock  
    signal (empty); // increment 'empty'  
} while(TRUE)
```

Classic Problems of Synchronization (The Readers-Writers Problem)

- A database is to be shared among several concurrent processes.
- Some of these processes may want **only to read the database**, whereas **others may want to update (that is, to read and write) the database**.
- We distinguish between these two types of processes by referring to the former as **Readers** and to the latter as **Writers**.
- Obviously, if **two readers access the shared data simultaneously, no adverse affects will result**.
- However, if a writer and some other thread (either a reader or a writer) access the database simultaneously, chaos may ensue.

To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database.

This synchronization problem is referred to as the readers-writers problem.

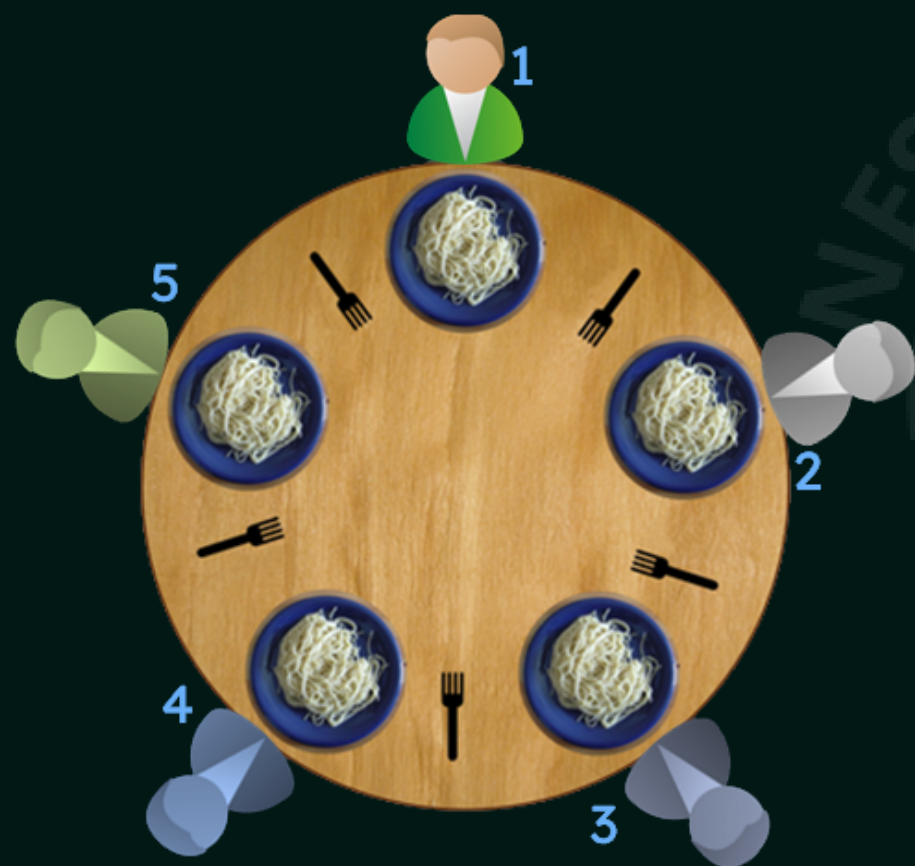
Solution to the Readers-Writers Problem using Semaphores:

We will make use of **two semaphores** and an **integer variable**:

1. **mutex**, a semaphore (**initialized to 1**) which is used to ensure mutual exclusion when **readcount** is updated i.e. when any reader enters or exit from the critical section.
2. **wrt**, a semaphore (**initialized to 1**) common to both reader and writer processes.
3. **readcount**, an integer variable (**initialized to 0**) that keeps track of how many processes are currently reading the object.

Writer Process	Reader Process
<pre>do { /* writer requests for critical section */ wait(wrt); /* performs the write */ // leaves the critical section signal(wrt); } while(true);</pre>	<pre>do { wait (mutex); readcnt++; // The number of readers has now increased by 1 if (readcnt==1) wait (wrt); // this ensure no writer can enter if there is even one reader signal (mutex); // other readers can enter while this current reader is // inside the critical section /* current reader performs reading here */ wait (mutex); readcnt--; // a reader wants to leave if (readcnt == 0) //no reader is left in the critical section signal (wrt); // writers can enter signal (mutex); // reader leaves } while(true);</pre>

Classic Problems of Synchronization (The Dining-Philosophers Problem)



Philosopher either

Thinks



When a philosopher thinks, he does not interact with her colleagues

Eats



When a philosopher gets hungry he tries to pick up the two forks that are closest to him (left & right). A philosopher may pick up only one fork at a time.

One cannot pick up a fork that is already in the hand of a neighbour.

When a hungry philosopher has both his forks at the same time, he eats without releasing his forks. When he has finished eating, he puts down both of his forks and starts thinking again.

One simple solution is to represent each fork/chopstick with a semaphore.

A philosopher tries to grab a fork/chopstick by executing a wait () operation on that semaphore.

He releases his fork/chopsticks by executing the signal () operation on the appropriate semaphores.

Thus, the shared data are

```
semaphore chopstick[5];
```

where all the elements of chopstick are initialized to 1.

The structure of philosopher i

```
do {  
wait (chopstick [i] ) ;  
wait(chopstick [ (i + 1) % 5] ) ;  
....  
// eat  
signal(chopstick [i]);  
signal(chopstick [(i + 1) % 5]);  
// think  
}while (TRUE);
```

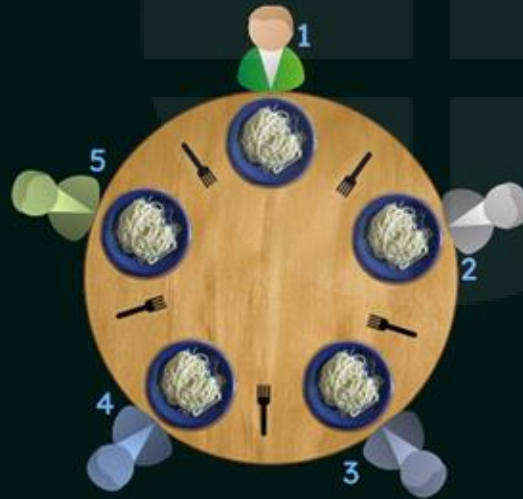
Although this solution guarantees that no two neighbors are eating simultaneously, it could still create a deadlock.

Suppose that all five philosophers become hungry simultaneously and each grabs their left chopstick. All the elements of chopstick will now be equal to 0.

When each philosopher tries to grab his right chopstick, he will be delayed forever.

Some possible remedies to avoid deadlocks:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up his chopsticks only if both chopsticks are available (to do this he must pick them up in a critical section).
- Use an asymmetric solution; that is, an odd philosopher picks up first his left chopstick and then his right chopstick, whereas an even philosopher picks up her right chopstick and then her left chopstick.



Monitors

- A high level abstraction that provides a convenient and effective mechanism for process synchronization.
- A monitor type presents a set of programmer-defined operations that provide mutual exclusion within the monitor.
- The monitor type also contains the **declaration of variables** whose values define the state of an instance of that type, along with the **bodies of procedures or functions that operate on those variables**.

Syntax of a Monitor

```
monitor monitor_name
{
// shared variable declarations
procedure P1 ( ... ) {
...
}
procedure P2 ( ... ) {
...
}
.
.
.
procedure Pn ( ... ) {
...
}
initialization code ( ... ) {
...
}
}
```

- A procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters.
- Similarly, the local variables of a monitor can be accessed by only the local procedures.
- **The monitor construct ensures that only one process at a time can be active within the monitor.**

Condition Construct-

condition x, y;

The only operations that can be invoked on a condition variable are **wait ()** and **signal ()**.

The operation **x.wait()** ; means that the process invoking this operation is suspended until another process invokes **x.signal ()**;

The **x. signal ()** operation resumes exactly one suspended process.

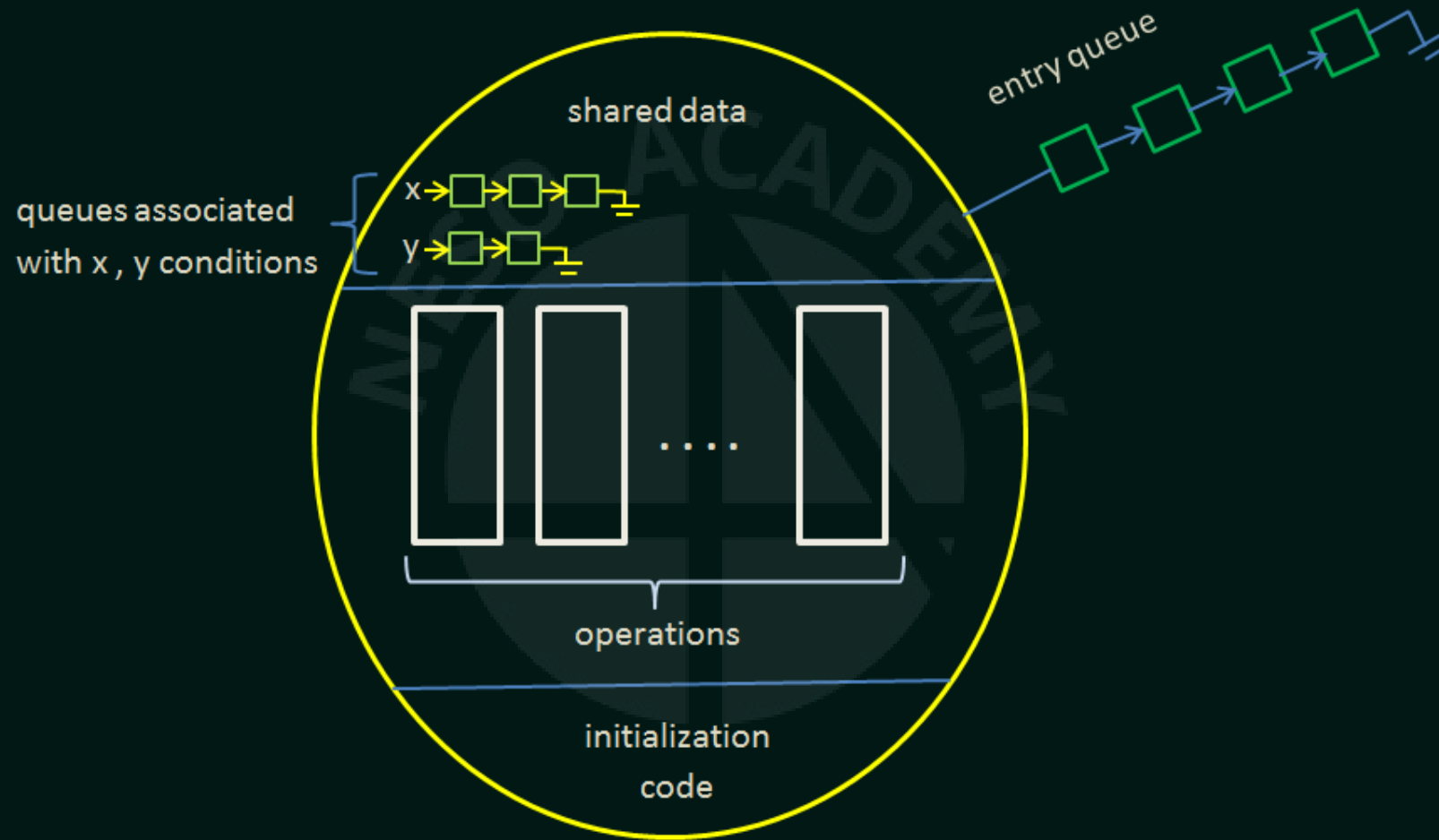


Fig: Schematic view of a monitor

Dining-Philosophers Solution Using Monitors

- We now illustrate monitor concepts by presenting a deadlock-free solution to the dining-philosophers problem.
- This solution imposes the restriction that a philosopher may pick up his chopsticks only if both of them are available.
- To code this solution, we need to distinguish among three states in which we may find a philosopher. For this purpose, we introduce the following data structure:

```
enum { thinking, hungry, eating } state [5] ;
```

- Philosopher i can set the variable $state[i] = eating$ only if his two neighbors are not eating: $(state[(i+4) \% 5] \neq eating)$ and $(state[(i+1)\% 5] \neq eating)$.
- We also need to declare `condition self [5] ;` where philosopher i can delay himself when he is hungry but is unable to obtain the chopsticks he needs.

A monitor solution to the dining-philosopher problem

```
monitor dp {  
    enum { THINKING, HUNGRY, EATING } state [5];  
    condition self [5];  
  
    void pickup (int i) {  
        state [i] = HUNGRY;  
        test (i);  
        if (state [i] != EATING)  
            self [i].wait();  
    }  
  
    void putdown(int i) {  
        state [i] = THINKING;  
        test ((i + 4) % 5);  
        test ((i + 1) % 5);  
    }  
  
    void test (int i) {  
        if ((state [(i + 4) % 5] != EATING) &&  
            (state [i] == HUNGRY) &&  
            (state [(i + 1) % 5] != EATING)) {  
            state [i] = EATING;  
            self [i].signal();  
        }  
    }  
  
    initialization-code () {  
        for (int i = 0; i < 5; i++)  
            state [i] = THINKING;  
    }  
}
```

Process Synchronization

Solved Problems (Part-1)

Question:

GATE 2010

Consider the methods used by processes P1 and P2 for accessing their critical sections whenever needed, as given below. The initial values of shared boolean variables S1 and S2 are randomly assigned.

Method Used by P1	Method Used by P2
while (S1 == S2) ; Critical Section S1 = S2;	while (S1 != S2) ; Critical Section S2 = not (S1);

Which one of the following statements describes the properties achieved?

- (a) Mutual exclusion but not progress
- (b) Progress but not mutual exclusion
- (c) Neither mutual exclusion nor progress
- (d) Both mutual exclusion and progress

Process Synchronization

Solved Problems (Part-2)

Question:

GATE 2010

The following program consists of 3 concurrent processes and 3 binary semaphores. The semaphores are initialized as $S_0=1$, $S_1=0$, $S_2=0$.

Process P0	Process P1	Process P2
<pre>while (true) { wait (S0); print (0); release (S1); release (S2); }</pre>	<pre>wait (S1); release (S0);</pre>	<pre>wait (S2); release (S0);</pre>

How many times will process P0 print '0' ?

- (a) At least twice
- (b) Exactly twice
- (c) Exactly thrice
- (d) Exactly once

Process Synchronization

Solved Problems (Part-3)

Question:

GATE 2007

Two processes, P1 and P2, need to access a critical section of code. Consider the following synchronization construct used by the processes:

Process P1	Process P2
<pre>while (true) { wants1 = true; while (wants2 == true); /* Critical Section */ wants1=false; } /* Remainder section */</pre>	<pre>while (true) { wants2 = true; while (wants1==true); /* Critical Section */ wants2 = false; } /* Remainder section */</pre>

Here, wants1 and wants2 are shared variables, which are initialized to false. Which one of the following statements is TRUE about the above construct?

- (a) It does not ensure mutual exclusion.
- (b) It does not ensure bounded waiting.
- (c) It requires that processes enter the critical section in strict alternation.
- (d) It does not prevent deadlocks, but ensures mutual exclusion.

Process Synchronization

Solved Problems (Part- 4)

Question:

GATE 2019

Consider three concurrent processes P1, P2 and P3 as shown below, which access a shared variable D that has been initialized to 100.

P1	P2	P3
.	.	.
.	.	.
$D = D + 20$	$D = D - 50$	$D = D + 10$
.	.	.
.	.	.

The process are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of D after the three processes have completed execution are X and Y respectively, then the value of Y-X is ____.

- (a) 80
- (b) 130
- (c) 50
- (d) None of these

Process Synchronization

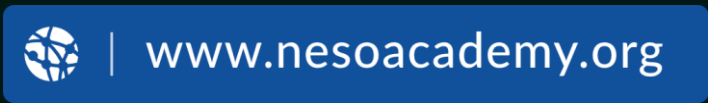
Solved Problems (Part-5)

Question:

GATE 1998

A counting semaphore was initialized to 10. Then 6 P (wait) operations and 4V (signal) operations were completed on this semaphore. The resulting value of the semaphore is

- (a) 0
- (b) 8
- (c) 10
- (d) 12



Neso Academy